OpenPiste Protocol

A proposal for modern open communication in fencing electronics

Status: Draft — working towards v1.0 **Author:** Piet Wauters **Repository:** https://github.com/OpenPiste **Website:** https://openpiste.org

---

Fencing electronics have long relied on two communication protocols: EFP1.1 (known as Cyrano), the dominant standard for communication between scoring apparatus and competition management software, and RS422-FPA, a serial protocol for driving external displays and scoreboards. Both were designed for their era and both served the community well. EFP1.1 has been in use since 2008. RS422-FPA traces its roots to 1995.

The world around them has changed. MQTT and JSON are now the lingua franca of connected devices. Libraries exist for every platform from microcontrollers to cloud services. The IoT ecosystem has solved, at scale, the same problems fencing electronics face: reliable message delivery, multiple subscribers, late-joining clients, structured extensible data. There is no longer a compelling reason to maintain a bespoke binary or CSV protocol when open, well-supported alternatives exist.

At the same time, there is a substantial installed base of EFP1.1-compatible apparatus and software. A new protocol that ignores this reality will not be adopted. Migration must be possible without requiring clubs and federations to replace working equipment overnight.

This document introduces the **OpenPiste Protocol**, a proposal for a modern, open communication standard for fencing electronics. It is structured in two levels:

Level 1 addresses the transition. It defines how existing EFP1.1 payloads can be transported over MQTT without any change to the payload itself. A bridge — a simple piece of software — relays messages between the existing UDP network and an MQTT broker. Existing apparatus and software require no modification. New MQTT-native subscribers (displays, piste monitors, video tools) can immediately consume live scoring data from existing infrastructure. Level 1 is not a long-term target. It is a practical bridge that allows the ecosystem to move at its own pace.

Level 2 is the destination. It defines a native JSON protocol designed from the ground up for MQTT, drawing on the field semantics of EFP1.1 and the message architecture of RS422-FPA while leaving behind the encoding constraints of both. It uses typed values, purpose-specific messages, retained state, and millisecond-precision timestamps. It is implementable on an ESP32 with standard open source libraries. It is designed to be genuinely open — any manufacturer, developer, club, or federation can implement it without restriction.

This is a working proposal, not a ratified standard. It is published in the hope that it will be useful, reviewed, and improved by the fencing electronics community. Comments, corrections, and contributions are welcome at https://github.com/OpenPiste.

OpenPiste Protocol — Level 2

Native JSON over MQTT

Status: Draft — working towards v1.0 **Date:** May 2026 **Author:** Piet Wauters **Protocol identifier:** `OPP2` **Repository:** https://github.com/OpenPiste **Website:** https://openpiste.org

Table of Contents

1. Introduction

Level 2 is the native JSON protocol of the OpenPiste platform. It is designed from the ground up for MQTT, taking full advantage of the broker's publish/subscribe model, topic hierarchy, and retained message capability. It does not carry forward the encoding constraints of EFP1.1.

Level 2 is intended to be a genuinely open standard — any apparatus manufacturer, software developer, club, or federation can implement it without restriction. The protocol identifier `OPP2` and a separate `version` field appear in every message, allowing receivers to identify the protocol family and enforce compliance rules appropriate for the declared version.

A JSON Schema for machine validation of all message types is maintained as a separate document in the OpenPiste repository. See `schemas/opp2/` at https://github.com/OpenPiste/protocol. *(Schema publication is a pending task — see Section 32.)*

---

2. Design principles

What follows are the goals OPP2 is designed around, not a list of implementation facts. Each one states the problem first — why MQTT, JSON, and topic-based routing were the right tools, not just the tools at hand, and why every one of them was chosen to keep independent implementers genuinely independent.

No component may assume vendor-specific behaviour from any other. Apparatus, scoreboards, remote controls, scoresheets, and the CMS itself are built by independent, uncoordinated implementers — a club may run one vendor's apparatus against a different vendor's scoreboard against a third party's CMS, several of them meeting for the first time on competition day. Nothing in OPP2 may quietly assume a specific implementation of any *other* component, including the broker itself: leaning on one broker's proprietary management API, one CMS's particular login scheme, or one apparatus's undocumented timing quirk would silently break that promise for whoever chose differently. This is, concretely, why the security model (Section 30) is built entirely from standard MQTT/TLS primitives — usernames and passwords, client certificates, ordinary broker ACLs — rather than any single broker's dynamic-user-management plugin: a mechanism only one vendor ships is not actually part of the protocol, however convenient it is in a single-vendor deployment. The same principle already governs the wire format itself (Section 1): any apparatus manufacturer, software developer, club, or federation can implement OPP2 without needing to coordinate with, or even be aware of, anyone else's implementation.

Designed to evolve without breaking what is already deployed. A fencing venue accumulates equipment purchased years apart, from vendors with no way to plan around each other's release schedules: apparatus fielded today must keep working when next year's scoreboard joins the same piste, and next year's apparatus must not confuse today's scoreboard. This is the actual reason OPP2 is JSON rather than a compact positional or binary encoding — every field is named, so a receiver silently ignores fields it doesn't recognise and a sender may omit a field a receiver doesn't require, with neither side needing to change (Section 29). New message types, new enumerated values, and new control commands (Section 24) are added the same way: additively, never by repurposing or renumbering something already in the field. Typed values — integers as integers, booleans as booleans, no numeric-string encoding — fall out of choosing JSON at all; they are a side effect of this decision, not the reason for it.

Purpose-specific messages. Each message type carries only the data relevant to its purpose. A scoreboard that only needs lights and scores does not need to parse fencer names or competition metadata. A video tool that only needs blade contact timestamps does not need to process clock ticks.

A subscriber sees correct state the instant it connects — never a blank slate. A scoreboard rebooting mid-bout, a video tool joining after the first touch, a display recovering from a Wi-Fi drop: none of these should show nothing, or stale nothing-yet, until the next event happens to fire, and none should need a bespoke "catch me up" handshake that only an OPP2-aware receiver would know to send. This is the actual reason state-bearing topics are published retained rather than as one-shot events — the broker itself hands a new or reconnecting subscriber the current value of every topic before anything else happens, with no periodic heartbeat resends and no bootstrap protocol to implement. It is also why a single, off-the-shelf MQTT broker is sufficient infrastructure at all: the platform already provides the guarantee OPP2 needs, instead of OPP2 needing to build one. A broker is a single point of failure — a well-understood property of MQTT deployments — and the ecosystem offers proven resilience options at whatever scale a venue needs. See Section 4.1.

Timestamps make time-critical events replayable. The lights, clock, and blade contact messages carry a millisecond UTC timestamp — no local time, no timezone offsets, no daylight saving adjustments. This is what makes accurate synchronisation with video replay systems possible at all, a capability absent from both EFP1.1 and RS422-FPA. See Section 28 for the encoding convention.

Built for a network that drops and duplicates, not against it. MQTT at QoS 1 guarantees delivery but not exactly-once delivery. Rather than layering exactly-once semantics on top, OPP2 accepts duplicates as a normal fact of the transport: every QoS 1 message carries a mandatory sequence counter (`seq`, Section 27) so a consumer can recognise and cheaply discard a redelivered duplicate, but nothing else in the protocol assumes duplicates can't happen.

Topic-based identity and routing. The piste identifier and the publisher's role — apparatus, software, remote, scoresheet, var — live in the MQTT topic and are never duplicated in the payload. A subscriber filters by piste or by publisher without parsing a single message body. This structure is also what makes the next principle possible at all: restricting who may write to a topic becomes an ordinary topic-pattern rule, the kind any standards-compliant broker already knows how to enforce — nothing OPP2-specific required on the broker side. See Section 5 for the topic structure.

Security scoped to the venue it protects. OPP2 assumes a physically-secured local network and a human present whenever a new component joins it — the same trust already implicit in wheeling a scoring apparatus onto a piste. It does not attempt bank-grade cryptographic trust between mutually suspicious parties, because that is not the threat a competition venue actually presents. What it does guarantee unconditionally: reading state is always open — any compliant subscriber sees the piste the moment it connects — while writing it is always authenticated, scoped to a publisher role using that same topic-pattern ACL, and revocable. See Section 30 for the full model.

Lightweight enough for the hardware that actually ships on a piste. A federation deploying OPP2 is buying many pistes, not one — apparatus is cost-sensitive, mass-produced, embedded hardware, not a server. This is as much a reason for JSON over MQTT as version compatibility is: both have small, free, well-proven libraries for constrained microcontrollers, and neither needs a code-generation step to implement. The reference implementation runs on an ESP32 using the Arduino MQTT client and ArduinoJson — nothing exotic, nothing licensed, nothing that doesn't fit in a few hundred kilobytes of flash.

---

3. Relationship to prior protocols

Level 2 draws on two existing protocols for its design:

EFP1.1 (Cyrano) provides the field semantics: state values, weapon codes, priority values, card counts, fencer status codes. These are preserved in Level 2 where they make sense, so developers familiar with EFP1.1 will recognise the values. The apparatus state machine defined in Section 25 is derived from EFP1.1 Section 4, adapted to the MQTT publish/subscribe model.

RS422-FPA (version 3.04a, 2019) provides the architectural inspiration for typed messages. RS422-FPA demonstrated that splitting scoring data into purpose-specific messages with different transmission priorities is practical and well-understood in the fencing community. In Level 2, MQTT topics replace the RS422 serial bus, the broker's retained message mechanism replaces RS422-FPA's periodic resend strategy, and QoS levels replace RS422-FPA's explicit message priority ordering.

RS422-FPA message	Level 2 topic	Notes
Msg 1 — lights	`lights`	Boolean fields; timestamp added
Msg 2 — clock	`clock`	Typed fields; timestamp added
Msg 3 — scores/cards	`score`	Integer fields; black card added
Msg 4 — status	`state` + `apparatus/connection`	Split into apparatus state and connection status
Msg 5+6 — competitor names	`fencers`	Restructured with left/right/common sections
Msg 7 — competition info	`match`	Match and competition metadata; round added
Msg 8 — UW2F	`uw2f`	Timer and P-cards
Msg 9 — bout control	`control`	Extensible command set
—	`blade_contact`	No RS422-FPA equivalent; blade contact with timestamp
—	`medical`	No RS422-FPA equivalent; medical timeout with countdown timer
—	`video_review`	No RS422-FPA equivalent; full call history and remaining counts
EFP1.1 HELLO	`software/connection`	Software presence announcement; retained

4. Network and Protocol Stack

4.1 Broker

Any MQTT 3.1.1 compliant broker. Mosquitto is recommended for club and competition use — it is open source, lightweight, and runs on a laptop or Raspberry Pi.

Broker resilience. Making the broker the single source of truth means that broker availability is important. This is a well-understood problem in MQTT deployments, and the ecosystem offers proven options at every scale. Implementers should choose the option appropriate for their context:

- **Persistent storage (all scales).** Any broker can be configured to persist retained messages and QoS 1 queues to disk. After a restart — whether planned or due to a crash — all retained state is restored immediately. For a competition on a dedicated host, a broker restart typically takes under a second and is transparent to connected clients, which reconnect automatically.
- **Active-passive failover (medium scale).** A standby broker instance on a second host, with shared or replicated persistent storage, can take over within seconds if the primary fails. Standard MQTT client libraries handle reconnection and session resumption automatically, so no changes are needed in any OPP2 device or software.
- **Native broker clustering (large scale).** Brokers such as EMQX, HiveMQ, and VerneMQ support horizontal clustering — multiple broker nodes share load and state, with no single point of failure. These are appropriate for national or international championships where infrastructure investment is justified.
- **Protocol-level resilience.** Even without infrastructure redundancy, the OPP2 protocol is designed to recover gracefully from a broker outage. The scoring apparatus maintains its own local state and continues to function during a brief outage. On reconnect, it republishes all retained topics (Section 25.3). The CMS republishes `software/fencers`, `software/match`, and `software/record` on reconnect. QoS 1 ensures that messages queued during the outage are delivered once connectivity is restored. A brief broker interruption mid-bout is an operational inconvenience, not a data loss event.

For most club and regional competition deployments, persistent storage on a single broker host — ideally with a UPS — provides sufficient resilience. Clustering is warranted for major international events where infrastructure investment matches the stakes.

4.2 Broker discovery

For club and small competition use, the broker host SHOULD be made discoverable via mDNS under the hostname:

```
openpiste.local
```

Any device on the local network can then reach the broker at `openpiste.local:1883` without IP address configuration. All OpenPiste-compatible devices SHOULD use this hostname as their default broker address, with fallback to a configurable IP address or hostname.

For larger competition setups with managed switches or multiple VLANs, mDNS may not propagate reliably across network boundaries. In these cases a static IP address or DHCP reservation for the broker is recommended, and the `openpiste.local` hostname may be configured in local DNS.

4.3 NTP

The broker host SHOULD also run a local NTP server. This allows all devices on the network to synchronise their clocks to UTC without requiring internet access. On Linux, `chrony` is recommended — it is lightweight and can serve NTP to local clients while itself operating without an upstream internet time source.

When all devices synchronise to the same local NTP server, timestamps in Level 2 messages are comparable across apparatus, displays, and video tools — enabling accurate video synchronisation on a fully self-contained competition network.

See Section 28 for the timestamp encoding convention, including the fallback behaviour when NTP is unavailable.

4.4 QoS

QoS	Applied to	Rationale
0 (at most once)	clock, blade_contact	High frequency or latency-critical. A missed clock tick self-corrects within one second. Blade contact retransmission latency would degrade timestamp precision for video sync.
1 (at least once)	all other topics	State changes and commands that must not be silently lost. QoS 1 may deliver duplicates — use the `seq` field to detect them (Section 27).

4.5 Retained messages

Apparatus-published state topics use retained messages. Software-published topics (`fencers` , `match` , `score` , `clock` , `uw2f`) do **not** use retained messages. `blade_contact` and `control` are also not retained.

Retained apparatus messages mean the broker holds the last published value for every apparatus topic. A subscriber connecting after the apparatus is online immediately receives the current state without waiting for the next publish cycle. Combined with QoS 1 on all state-bearing topics, this eliminates the need for periodic heartbeat resends.

fencers and match (publisher: software) are **not retained**. The apparatus is the authoritative source of truth for what is currently happening on the piste. If `software/fencers` and `software/match` were retained, a stale assignment from a previous session could be replayed to a newly connected apparatus when no live CMS is present. The apparatus cannot distinguish a retained message from a live one and would have no way to know whether the assignment is current. Making these non-retained means the apparatus only accepts fencer and match data when a CMS is actively pushing it.

`apparatus/fencers` (Section 15) is retained, same as every other apparatus-published topic — the apparatus is authoritative, and a late-joining CMS needs its current assignment immediately on reconnect, exactly like `apparatus/score` .

`software/score` is **not retained**, for the same reason as `fencers` and `match` : the apparatus is the sole executor and authority for score, cards, and priority (Section 13), and already must hold this state to drive its own display and enforce the clock interlock. A `software/score` message is a correction pushed *to* the apparatus, not a fact the apparatus should trust arrived unattended — a stale retained correction replayed to a newly connected apparatus with no live CMS behind it could silently overwrite a score the apparatus

has been tracking correctly on its own since the message was first published. Making it non-retained means the apparatus only applies a software-originated correction when a CMS is actively publishing it, exactly like `fencers` and `match`. `apparatus/score` is unaffected by this and remains retained, same as every other apparatus-published topic.

`software/clock` is **not retained**, for the same reason: the apparatus is the sole real-time authority for its own stopwatch, running start/stop and the clock interlock locally regardless of network presence (Section 11). A `software/clock` message only seeds or corrects the stopwatch's starting value — a one-shot instruction, not a fact the apparatus should adopt unattended from a stale retained replay. A retained `software/clock` left over from an earlier relay reset or piste transfer could otherwise be replayed to a newly connected apparatus and silently reset a clock it has been running correctly on its own since. `apparatus/clock` is unaffected and remains retained and QoS 0, unchanged.

`software/uw2f` is **not retained**, for the identical reason: the apparatus holds UW2F timer and P-card state continuously for its own display (Section 19), and `software/uw2f` is a one-shot seed or correction pushed to it — used, for example, when a mid-bout piste transfer needs a new apparatus's passivity timer and P-cards to start from where the old apparatus left off rather than from zero. `apparatus/uw2f` is unaffected and remains retained.

Connection recovery follows this hierarchy:

1. If the apparatus retains its RAM state (network glitch, no power loss), it republishes its own retained topics on reconnect. No CMS action is needed.
2. If the apparatus reboots, it first reloads state from its own non-volatile memory. Failing that, it reads its own last-known state from the retained apparatus topics on the broker.
3. Only if neither local nor broker apparatus state is recoverable does the apparatus publish a `NEXT` control command, prompting the CMS to republish `fencers` and `match`.

blade_contact is not retained because it is a point-in-time event. A retained blade contact message would cause a late subscriber to receive a contact notification with no way to know it was already resolved.

control is not retained because commands are one-shot. A late subscriber must not act on a BEGIN or NEXT command that was issued before it connected.

software/record and scoresheet/record are **retained**. Both are consumed by display components — scoresheets, monitors, scoreboards — that benefit from immediate delivery of the current state on connect. Unlike `fencers` and `match`, neither targets the scoring apparatus, so there is no risk of a stale retained message triggering incorrect apparatus behaviour. `software/record` is retained because display components connecting mid-slot need the full bout context immediately, without waiting for the next bout transition. `scoresheet/record` is retained because the broker serves as the scoresheet's persistent annotation memory, surviving reconnects, reboots, and piste transfers.

scoresheet/event is **not retained**. It is a per-event notification, not a state description. A retained event message would deliver a single annotation to late subscribers with no preceding context — misleading rather than helpful. The full annotation history is always available in `scoresheet/record`.

4.6 Last Will and Testament

Both the apparatus and competition management software MUST configure a Last Will and Testament (LWT) message when connecting to the broker. The LWT is set in the MQTT CONNECT packet and is published automatically by the broker on unexpected disconnection.

Apparatus LWT:

- **Topic:** `openpiste/{piste_id}/apparatus/connection`
- **Payload:** `{"online": false}`
- **QoS:** 1 — **Retain:** true

Software LWT:

- **Topic:** `openpiste/{piste_id}/software/connection`
- **Payload:** `{"online": false}`
- **QoS:** 1 — **Retain:** true

Why the offline payload is minimal. The online and offline forms of `connection` are deliberately asymmetric — this is not an oversight or a leftover from EFP1.1. A Will message is fixed in the CONNECT packet before the session begins and cannot be updated afterward without reconnecting, so any field baked into it reflects state at connect time, not at the moment of actual disconnection. Rather than let a subscriber guess which frozen fields are still trustworthy once a component drops offline, the offline payload carries only `online: false`: nothing else about a disconnected component should be treated as current. The same reasoning applies to `var/connection` (Section 23), which is structured identically.

The apparatus watches `openpiste/{piste_id}/software/connection` to determine whether a live CMS is present. See Section 25 for how this affects apparatus behaviour.

4.7 Port

Standard MQTT port 1883 (unencrypted) or 8883 (TLS).

---

5. Topic structure

All Level 2 topics follow this pattern:

```
openpiste/{piste_id}/{publisher}/{message_type}
```

Segment	Description	Examples
`openpiste`	Fixed platform prefix	—
`{piste_id}`	Piste identifier — number, name, or colour	`17`, `podium`, `rouge`, `vert`
`{publisher}`	Role of the publishing device — see values below	`apparatus`, `software`, `remote`
`{message_type}`	Message type as defined in Section 6	`lights`, `clock`, `score`

Publisher values:

Value	Meaning
`apparatus`	Message published by the scoring apparatus
`software`	Message published by competition management software
`remote`	Message published by a remote control device
`var`	Message published by the video referee system
`scoresheet`	Message published by an electronic score sheet (tablet or smartphone used by the table official)

The `{piste_id}` and `{publisher}` segments in the topic are the **authoritative** sources of piste identity and publisher role respectively. Neither is duplicated in the payload. Consumers that need to store these values alongside the payload MUST extract them from the topic at the time of receipt.

Encoding the publisher role in the topic allows subscribers to filter by publisher at the broker level with no payload parsing required. It also maps directly onto broker access control: each publisher role can be restricted to writing only within its own topic namespace (see Section 30).

Useful subscription patterns:

```
openpiste/#                          # all messages from all pistes
openpiste/17/#                       # all messages from piste 17
openpiste/+/apparatus/#              # all apparatus messages from all pistes
openpiste/+/apparatus/lights         # lights from all pistes
openpiste/+/apparatus/score          # scores from all pistes
openpiste/+/+/control                # control events from all publishers, all pistes
openpiste/+/apparatus/connection     # apparatus connection status from all pistes
openpiste/+/software/connection      # software connection status from all pistes
```

---

6. Message overview

Topic	Publisher	QoS	Retained	Published when
`apparatus/connection`	apparatus	1	Yes	On connection or disconnection (including LWT)
`software/connection`	software	1	Yes	On connection or disconnection (including LWT)
`apparatus/lights`	apparatus	1	Yes	On any light change
`apparatus/clock`	apparatus	0	Yes	Every second while running; on any clock state change
`software/clock`	software	1	No	On clock seeding or correction pushed to the apparatus (relay reset, piste transfer)
`apparatus/blade_contact`	apparatus	0	No	On blade contact event
`apparatus/score`	apparatus	1	Yes	On score, card, or priority change
`software/score`	software	1	No	On score, card, or priority correction pushed to the apparatus
`apparatus/state`	apparatus	1	Yes	On apparatus state change
`software/fencers`	software	1	No	On fencer, coach, or referee identity change
`apparatus/fencers`	apparatus	1	Yes	On a referee-initiated left/right swap for the active bout
`software/match`	software	1	No	On match or competition metadata change
`software/record`	software	1	Yes	On slot assignment, bout confirmation, confirmed fencer swap, or piste transfer
`apparatus/uw2f`	apparatus	1	Yes	On UW2F timer or P-card change
`software/uw2f`	software	1	No	On UW2F timer or P-card seeding/correction pushed to the apparatus (piste transfer)
`apparatus/medical`	apparatus	1	Yes	On medical timeout event or timer update
`apparatus/video_review` or `var/video_review`	apparatus or var	1	Yes	On video review request or resolution
`apparatus/control`, `software/control`, `remote/control`, `var/control`, or `scoresheet/control`	apparatus, software, remote, var, or scoresheet	1	No	On remote control event
`scoresheet/event`	scoresheet	1	No	On each new table official annotation (fire-and-forget; see `scoresheet/record` for accumulated state)
`scoresheet/record`	scoresheet	1	Yes	Accumulated annotation log for the current slot; updated after

Topic	Publisher	QoS	Retained	Published when
				each new annotation

---

7. Common fields

Every Level 2 message contains the following common fields. They appear first in every payload.

Field	Type	QoS 0	QoS 1	Description
`protocol`	string	Mandatory	Mandatory	Always `"OPP2"`
`version`	string	Mandatory	Mandatory	Protocol version — e.g. `"1.0"`. See Section 29.
`seq`	integer	Absent	Mandatory	Global sequence counter — see Section 27
`ts`	integer	Mandatory	Recommended	Timestamp — see Section 28

`ts` is mandatory on QoS 0 messages (clock, blade_contact) and on control messages. It is recommended on all other QoS 1 messages.

Publisher identity is encoded in the topic's `{publisher}` segment and is not present in the payload. See Section 5 for the rationale.

---

8. Message: apparatus/connection

Topic: `openpiste/{piste_id}/apparatus/connection` **QoS:** 1 **Retained:** Yes

Indicates whether the scoring apparatus is currently connected to the broker. The apparatus publishes this message on successful connection. The broker publishes the LWT payload automatically on unexpected disconnection.

Payload — apparatus online

```
{
  "protocol":   "OPP2",
  "version":    "1.0",
  "seq":        1,
  "online":     true,
  "device":     "OpenPiste-ESP32",
  "fw_version": "1.0.0"
}
```

Payload — offline (LWT, published by broker)

Deliberately minimal, not symmetric with the online payload — see "Why the offline payload is minimal" in Section 4.6.

```
{
  "online": false
}
```

Fields

Field	Type	M/O	Description
`protocol`	string	M	Always `"OPP2"` (omitted in LWT)
`version`	string	M	Protocol version (omitted in LWT)
`seq`	integer	M	Global sequence counter (omitted in LWT)
`online`	boolean	M	`true` — connected; `false` — offline
`device`	string	O	Device model or identifier
`fw_version`	string	O	Firmware version

---

9. Message: software/connection

Topic: `openpiste/{piste_id}/software/connection` **QoS:** 1 **Retained:** Yes

Indicates whether competition management software is currently connected to the broker and active for this piste. This is the OPP2 equivalent of the EFP1.1 HELLO message. The software publishes this message on connection; the broker publishes the LWT payload on unexpected disconnection.

The apparatus watches this topic to determine whether a live CMS is present. When the apparatus sees `"online": true` after a reboot or network recovery, it knows a CMS is available and may publish a `NEXT` control command to request match data if its own state is not recoverable locally. See Section 25.

Unlike the EFP1.1 HELLO — which was a periodic heartbeat every 15 seconds and served to trigger a full INFO resend from the apparatus — the OPP2 `software/connection` message is published only on connection state change. The need for a periodic trigger is eliminated by the retained message mechanism: the broker always holds the current apparatus state and delivers it to any subscriber immediately on connection.

Payload — software online

```
{
  "protocol":    "OPP2",
  "version":     "1.0",
  "seq":         1,
  "online":      true,
  "software":    "EnGarde",
  "sw_version":  "12.1"
}
```

Payload — offline (LWT, published by broker)

Deliberately minimal, not symmetric with the online payload — see "Why the offline payload is minimal" in Section 4.6.

```
{
  "online": false
}
```

Fields

Field	Type	M/O	Description
`protocol`	string	M	Always `"OPP2"` (omitted in LWT)
`version`	string	M	Protocol version (omitted in LWT)
`seq`	integer	M	Global sequence counter (omitted in LWT)
`online`	boolean	M	`true` — connected and active; `false` — offline
`software`	string	O	Software name or identifier
`sw_version`	string	O	Software version

---

10. Message: lights

Topic: `openpiste/{piste_id}/apparatus/lights` **QoS:** 1 **Retained:** Yes

Published immediately on any change to the light state. This is the highest-priority message — published before any other pending message when a light state changes. QoS 1 ensures a missed lights message is retransmitted, preventing subscribers from holding a permanently incorrect light state.

Light colour conventions apply across all weapons:

- **Red** light: left fencer scored (on target)
- **Green** light: right fencer scored (on target)
- **White** light: off-target hit (foil) or broken circuit (sabre)

Payload

```
{
  "protocol": "OPP2",
  "version":  "1.0",
  "seq":      42,
  "ts":       1715539200123,
  "right": {
    "green": false,
    "white": true
  },
  "left": {
    "red":   true,
    "white": false
  }
}
```

Fields

Field	Type	M/O	Default	Description
`protocol`	string	M	—	Always `"OPP2"`
`version`	string	M	—	Protocol version
`seq`	integer	M	—	Global sequence counter
`ts`	integer	M	—	Timestamp of light change — see Section 28
`right.green`	boolean	M	`false`	Right fencer on-target light
`right.white`	boolean	M	`false`	Right fencer white (off-target / broken circuit) light
`left.red`	boolean	M	`false`	Left fencer on-target light
`left.white`	boolean	M	`false`	Left fencer white (off-target / broken circuit) light

---

11. Message: clock

Topic: `openpiste/{piste_id}/{publisher}/clock` **QoS:** `apparatus/clock` — 0. `software/clock` — 1.
Retained: `apparatus/clock` — Yes. `software/clock` — No, see Section 4.5 for rationale.

The apparatus publishes `apparatus/clock` once per second while the stopwatch is running, and immediately on any clock state change (start, stop, reset). QoS 0 is appropriate for the running stream — a missed tick self-corrects within one second, since the next tick supersedes it regardless of delivery. Retention addresses a different gap: while the clock is stopped (idle between bouts, potentially for minutes), there is no periodic republish, so a newly-connecting or reconnecting subscriber needs the retained value to see the correct paused time immediately rather than wait for the next state change — the same bootstrap need that justifies retention on `apparatus/score`.

Competition management software publishes `software/clock` to seed or correct the apparatus's stopwatch to a specific value — a one-shot instruction, not a repeating stream, so it is QoS 1 rather than QoS 0. Two cases: resetting a fresh team relay's clock to its starting value (e.g. `3:00`), and seeding a new apparatus's elapsed time when an in-progress bout is moved to a different piste mid-match (apparatus failure, scheduling) so the clock survives the move instead of restarting from zero. `software/clock` only sets the stopwatch's starting value; the apparatus resumes its own authoritative start/stop and interlock logic from that point — it never bypasses or replaces the apparatus's own real-time tracking.

Payload — apparatus/clock

```
{
  "protocol": "OPP2",
  "version":  "1.0",
  "ts":       1715539200123,
  "running":  true,
  "time_ms":  89250,
  "time":     "1:29.25"
}
```

Payload — software/clock

```
{
  "protocol": "OPP2",
  "version":  "1.0",
  "seq":      12,
  "ts":       1715539200123,
  "running":  false,
  "time_ms":  89250,
  "time":     "1:29.25"
}
```

Fields

Field	Type	M/O	Default	Description
`protocol`	string	M	—	Always `"OPP2"`
`version`	string	M	—	Protocol version
`seq`	integer	M on `software/clock`; absent on `apparatus/clock`	—	Global sequence counter — see Section 27. `apparatus/clock` is QoS 0, where `seq` is omitted per Section 7; `software/clock` is QoS 1, where it is mandatory.
`ts`	integer	M	—	Timestamp of this publication — see Section 28
`running`	boolean	M	`false`	`true` if the stopwatch is currently running. On `software/clock` this **MUST** be `false`. Starting and stopping the clock is a real-time function driven by the apparatus's own interlock (hit sensing, referee start/stop) — never a network-commanded one. A `software/clock` message only seeds the paused starting value; the referee resumes it locally. An apparatus receiving `software/clock` with `"running": true` MUST NOT start its clock from this field — it MUST treat the message as if `running` were `false` (seed the value, stay paused), and SHOULD log the malformed message.
`time_ms`	integer	M	`0`	Stopwatch value in milliseconds — current value on `apparatus/clock`, seeded value on `software/clock`
`time`	string	M	`"0:00"`	Formatted as `"M:SS"` or `"M:SS.cc"`. Hundredths mandatory below 10 seconds.

Note: `seq` is absent on QoS 0 messages.

---

12. Message: blade_contact

Topic: `openpiste/{piste_id}/apparatus/blade_contact` **QoS:** 0 **Retained:** No

Published on blade contact events. The primary purpose of this message is to provide a precise timestamp for synchronisation with video replay systems. Not every blade contact is a scoring touch or a parry — this message records the raw electrical event. It enables referees and AI tools to determine whether a scoring action involved genuine blade contact.

QoS 0 is used because retransmission latency would degrade timestamp precision, which is the primary value of this message.

Semantics: stateful on/off, not momentary. `blade_contact` is published once when contact begins (`active: true`) and once when it clears (`active: false`) — two events per contact, not a single momentary publish. This is a deliberate choice: it gives video and AI review applications a precise contact *duration*, not just an instant, at the cost of roughly double the event volume. Implementers should expect these events to arrive in **bursts** — a rapid sequence of parries produces a corresponding burst of paired on/off transitions in quick succession, not a steady low rate — and should design accordingly (e.g. don't

assume even spacing between events). Not retained, regardless of this stateful shape: QoS 0 means an `active: true` message's corresponding `false` could be lost in transit, and a retained message would then leave a late subscriber stuck believing contact is still active with no way to know otherwise.

Payload

```
{
  "protocol": "OPP2",
  "version":  "1.0",
  "ts":       1715539200089,
  "active":   true
}
```

Fields

Field	Type	M/O	Description
`protocol`	string	M	Always `"OPP2"`
`version`	string	M	Protocol version
`ts`	integer	M	Timestamp of contact event — see Section 28
`active`	boolean	M	`true` — blade contact detected; `false` — contact cleared

Note: `seq` is absent on QoS 0 messages.

---

13. Message: score

Topic: `openpiste/{piste_id}/{publisher}/score` **QoS:** 1 **Retained:** `apparatus/score` — Yes. `software/score` — No, see Section 4.5 for rationale.

Published on any change to scores, cards, or priority. The apparatus publishes under `apparatus/score`; competition management software correcting a score publishes under `software/score`. All subscribers see both; the publisher segment identifies the origin.

The apparatus holds score/card/priority state for its own display and clock-interlock regardless of network presence, so `software/score` is a correction pushed to it, not a fact the apparatus should adopt unattended from a stale retained replay. On reconnect, a CMS that needs to re-assert a correction re-publishes it live, the same pattern already used for `software/fencers` and `software/match`. Its case is the same as `software/clock`'s and `software/uw2f`'s piste-transfer use (Sections 11, 19): when an in-progress bout moves to a different apparatus mid-match, the new apparatus is seeded with the old one's score, cards, and priority so they survive the move instead of resetting to zero.

Payload

```
{
  "protocol": "OPP2",
  "version":  "1.0",
  "seq":      43,
  "right": {
    "score":       8,
    "status":      "V",
    "yellow_card": false,
    "red_cards":   1,
    "black_card":  false
  },
  "left": {
    "score":       6,
    "status":      "D",
    "yellow_card": false,
    "red_cards":   0,
    "black_card":  false
  },
  "priority": "N"
}
```

Fields

Field	Type	M/O	Default	Description
`protocol`	string	M	—	Always `"OPP2"`
`version`	string	M	—	Protocol version
`seq`	integer	M	—	Global sequence counter
`right.score`	integer	M	`0`	Right fencer score
`right.status`	string	M	`"U"`	Right fencer match status — see values below
`right.yellow_card`	boolean	M	`false`	Right fencer yellow card
`right.red_cards`	integer	M	`0`	Right fencer red card count (0–9)
`right.black_card`	boolean	M	`false`	Right fencer black card
`left.score`	integer	M	`0`	Left fencer score
`left.status`	string	M	`"U"`	Left fencer match status
`left.yellow_card`	boolean	M	`false`	Left fencer yellow card
`left.red_cards`	integer	M	`0`	Left fencer red card count (0–9)
`left.black_card`	boolean	M	`false`	Left fencer black card
`priority`	string	M	`"N"`	`"N"` none, `"R"` right, `"L"` left

Status values:

Value	Meaning
`"U"`	Undefined
`"V"`	Victory
`"D"`	Defeat
`"A"`	Abandonment
`"E"`	Exclusion
`"DNS"`	Did not show

---

14. Message: state

Topic: `openpiste/{piste_id}/apparatus/state` **QoS:** 1 **Retained:** Yes

Indicates the current operational state of the scoring apparatus. Published on every state transition. See Section 25 for the full state machine.

Payload

```
{
  "protocol": "OPP2",
  "version":  "1.0",
  "seq":      44,
  "state":    "F"
}
```

Fields

Field	Type	M/O	Default	Description
`protocol`	string	M	—	Always `"OPP2"`
`version`	string	M	—	Protocol version
`seq`	integer	M	—	Global sequence counter
`state`	string	M	`"W"`	Apparatus state — see values below

State values (inherited from EFP1.1):

Value	Meaning
`"F"`	Fencing — stopwatch running
`"H"`	Halt — stopwatch stopped, bout in progress
`"P"`	Pause — between periods
`"W"`	Waiting — no active bout
`"E"`	Ending — awaiting ACK from software

15. Message: fencers

Topic: `openpiste/{piste_id}/software/fencers` or `openpiste/{piste_id}/apparatus/fencers` **QoS:** 1 **Retained:** `software/fencers` — No, see Section 4.5 for rationale. `apparatus/fencers` — Yes, per the default for apparatus-published topics (Section 4.5).

Published by software when any participant identity information changes — this is the authoritative assignment, and the only one apparatus and Cyrano-compatible systems consult on load. In team competitions, republished at the end of each round when fencer assignments change.

`apparatus/fencers` exists primarily for one purpose: FIE Technical Rules t.22 requires a left-handed fencer to stand on the referee's left when fencing a right-hander, regardless of call order, and the referee at the piste always has final say over software's assignment. When the referee corrects the assignment — via a button on the apparatus, or a remote control, or any other locally-implemented mechanism entirely outside OPP2's scope (see the OpenPisteRemoteControl subspec) — the apparatus republishes `apparatus/fencers` with the `left` and `right` fencer objects exchanged, verbatim, from what it last received via `software/fencers`. Publishing the full fencer objects (not just the two ids) makes the intent unambiguous: this is a deliberate identity exchange, not a partial or corrupted update. There is no dedicated control command for this — the local correction is invisible to every other subscriber, exactly like a button press or IR remote signal; only its effect, this message, is ever visible on the broker.

The apparatus MAY also republish `apparatus/fencers` for reasons unrelated to a swap — confirming a freshly-received `software/fencers` verbatim, republishing its last-known state on MQTT reconnect, or clearing to empty (`fencer.present: false` on both sides) once identifying data is no longer relevant between bouts. None of these are errors; software MUST distinguish them from a genuine swap by comparing content, not by assuming every `apparatus/fencers` publish means a swap happened.

Software's responsibility on receiving `apparatus/fencers`: compare `left.fencer.id` and `right.fencer.id` against what it currently has on file for the active bout on that piste.

- **Empty** (both sides absent): the normal idle-between-bouts state. Ignore.
- **Identical to the current assignment** (same two ids, same sides): a confirmation echo, not a swap. Ignore — nothing to apply, nothing to flag.
- **Clean swap** (the same two ids, exchanged): apply it — exchange the bout's own left/right assignment, including any already-recorded score and card data, so it stays attached to the correct fencer rather than to whichever column it was previously stored under (see `Bout.swapSides`, `services/bouts.js`) — then re-publish `software/fencers` and `software/record` (same `slot_id`, same `bouts[n].id`, `left` / `right` exchanged — see Section 17) so every other subscriber converges on the same corrected assignment. Section 18 covers what a scoresheet does with this.
- **Anything else** (one id unchanged and the other different, both different, or any other pairing that is neither identical nor a clean exchange of the current two; or no active bout on that piste matches at all; or the matching bout already has a result): do **not** apply it. Log the mismatch and surface a clear, immediate warning to the director — this should never happen in practice, and treating it as fact rather than flagging it risks silently misattributing a result. If the apparatus subsequently publishes `apparatus/control` `"END"` while the mismatch remains unresolved, software MUST NAK it (see Section 25.4) rather than register a result against an unconfirmed fencer assignment.

The message is structured in three sections: `left`, `right`, and `common`.

Payload

```
{
  "protocol": "OPP2",
  "version":  "1.0",
  "seq":      45,
  "left": {
    "fencer": {
      "id":        "32",
      "name":      "B. Panini",
      "nation":    "ITA",
      "club":      "Club Scherma Roma",
      "club_abbr": "CSR"
    },
    "coach": {
      "id":     "c1",
      "name":   "M. Rossi",
      "nation": "ITA"
    }
  },
  "right": {
    "fencer": {
      "id":        "28",
      "name":      "P. Martin",
      "nation":    "FRA",
      "club":      "Cercle d'Escrime de Paris",
      "club_abbr": "CEP"
    },
    "coach": {
      "id":     "c2",
      "name":   "J. Dupont",
      "nation": "FRA"
    }
  },
  "common": {
    "referee": {
      "id":     "132",
      "name":   "J. Smith",
      "nation": "GBR"
    }
  }
}
```

`common.referee` names the piste's primary referee — the only officiating identity the apparatus (and Cyrano-compatible systems) needs to know about. A bout may have additional officials assigned — a second referee, a video official, assessors — but those are a scoresheet/display concern, not an apparatus one, and are published in `software/record` (Section 17) instead.

Fields

Field	Type	M/O	Description
`protocol`	string	M	Always `"OPP2"`
`version`	string	M	Protocol version
`seq`	integer	M	Global sequence counter
`left.fencer.id`	string	M	Left fencer identifier
`left.fencer.name`	string	M	Left fencer name
`left.fencer.nation`	string	M	IOC 3-letter nation code
`left.fencer.club`	string	O	Left fencer club name
`left.fencer.club_abbr`	string	O	Left fencer club abbreviation — for display in space-constrained contexts
`left.coach.id`	string	O	Left fencer coach identifier
`left.coach.name`	string	O	Left fencer coach name
`left.coach.nation`	string	O	Left fencer coach nation
`right.fencer.id`	string	M	Right fencer identifier
`right.fencer.name`	string	M	Right fencer name
`right.fencer.nation`	string	M	IOC 3-letter nation code
`right.fencer.club`	string	O	Right fencer club name
`right.fencer.club_abbr`	string	O	Right fencer club abbreviation — for display in space-constrained contexts
`right.coach.id`	string	O	Right fencer coach identifier
`right.coach.name`	string	O	Right fencer coach name
`right.coach.nation`	string	O	Right fencer coach nation
`common.referee.id`	string	O	Referee identifier
`common.referee.name`	string	O	Referee name
`common.referee.nation`	string	O	Referee nation

Optional fields SHOULD be omitted when not available. Receivers MUST handle their absence gracefully.

---

16. Message: match

Topic: `openpiste/{piste_id}/software/match` **QoS:** 1 **Retained:** No — see Section 4.5 for rationale.

Published by software when match or competition metadata changes, including round changes during team competitions.

Payload

```
{
  "protocol":    "OPP2",
  "version":     "1.0",
  "seq":         46,
  "weapon":      "E",
  "type":        "I",
  "competition": "efj-eq",
  "phase_type":  "DE",
  "phase":       "3",
  "poule":       "A32",
  "match":       12,
  "round":       1,
  "scheduled":   "13:15"
}
```

Fields

Field	Type	M/O	Default	Description
`protocol`	string	M	—	Always `"OPP2"`
`version`	string	M	—	Protocol version
`seq`	integer	M	—	Global sequence counter
`weapon`	string	M	—	`"F"` foil, `"E"` épée, `"S"` sabre
`type`	string	M	—	`"I"` individual, `"T"` team
`competition`	string	M	—	Competition identifier
`phase_type`	string	M	—	Phase type — see values below
`phase`	string	M	—	Phase identifier
`poule`	string	M	—	Poule or tableau identifier
`match`	integer	M	—	Match number
`round`	integer	M	`1`	Current round or period (team: 1–9; individual: 1–3)
`scheduled`	string	O	—	Scheduled start time as `"HH:MM"`

Phase type values:

Value	Meaning
`"pool"`	Pool / poule round
`"DE"`	Direct elimination
`"repechage"`	Repechage
`"classification"`	Classification round

Additional phase type values may be defined in future revisions without a protocol version change.

17. Message: software/record

Topic: `openpiste/{piste_id}/software/record` **QoS:** 1 **Retained:** Yes

Published by the competition management software to describe the full context of the current piste assignment: the ordered list of participants, the list of bouts with their identities and results so far, and the `slot_id` that ties this record to the scoresheet's annotation log.

A *slot* is the unit of work assigned to a piste for a given session — a pool round or a range of DE bouts. `software/record` is published when a slot is first assigned (bouts listed, no results yet), updated after each bout is confirmed (ACK received), and republished in full on a piste transfer.

Unlike `software/fencers` and `software/match` — which target the scoring apparatus and describe only the active bout — `software/record` targets display components (scoresheets, scoreboards, monitors) and describes the entire slot. The apparatus does not subscribe to this topic.

`software/record` is also the canonical home for the slot's full officiating roster — the primary referee plus any second referee, video official, or assessors assigned to it. `software/fencers` carries only the single `common.referee` field the apparatus needs (Section 15); everything else about who's officiating belongs here, since it is scoresheet/display-facing and retained, so a reconnecting scoresheet gets the full roster immediately rather than waiting for the next apparatus-facing publish.

Retained rationale: `software/fencers` and `software/match` are not retained because a stale assignment could mislead the apparatus on reconnect (see Section 4.5). `software/record` does not carry this risk — display components can render the last-known state immediately and update incrementally on each new message. Retained delivery means a scoresheet or monitor connecting mid-slot receives the full slot context without waiting for the next bout transition.

Payload

```
{
  "protocol":    "OPP2",
  "version":     "1.0",
  "seq":         53,
  "slot_id":     "f47ac10b-58cc-4372-a567-0e02b2c3d479",
  "phase_type":  "pool",
  "label":       "Pool A",
  "active_bout": 3,
  "referee": {
    "id":     "132",
    "name":   "J. Smith",
    "nation": "GBR"
  },
  "referee2": {
    "id":     "145",
    "name":   "K. Andersson",
    "nation": "SWE"
  },
  "video_official": {
    "id":     "ref002",
    "name":   "L. Dubois",
    "nation": "FRA"
  },
  "assessor1": {
    "id":     "88",
    "name":   "R. Kim",
    "nation": "KOR"
  },
  "participants": [
    { "position": 1, "id": "32", "name": "B. Panini", "nation": "ITA", "club_abbr": "CSR" },
    { "position": 2, "id": "28", "name": "P. Martin", "nation": "FRA", "club_abbr": "CEP" },
    { "position": 3, "id": "15", "name": "A. Koch",   "nation": "GER", "club_abbr": "BFC" }
  ],
  "bouts": [
    {
      "id": 1,
      "left":   { "id": "32", "name": "B. Panini", "nation": "ITA" },
      "right":  { "id": "28", "name": "P. Martin", "nation": "FRA" },
      "result": { "left_score": 5, "right_score": 3, "left_status": "V", "right_status": "D" }
    },
    {
      "id": 2,
      "left":   { "id": "15", "name": "A. Koch",   "nation": "GER" },
      "right":  { "id": "32", "name": "B. Panini", "nation": "ITA" },
      "result": null
    },
```

```
    {
      "id": 3,
      "left":   { "id": "28", "name": "P. Martin", "nation": "FRA" },
      "right":  { "id": "15", "name": "A. Koch",   "nation": "GER" },
      "result": null
    }
  ]
}
```

Fields

Field	Type	M/O	Default	Description
`protocol`	string	M	—	Always `"OPP2"`
`version`	string	M	—	Protocol version
`seq`	integer	M	—	Global sequence counter
`slot_id`	string	M	—	Opaque slot identifier generated by the CMS when the slot is assigned to a piste. Unchanged on piste transfer. CMS implementations SHOULD use UUID v4.
`phase_type`	string	M	—	Phase type — same values as in `software/match`
`label`	string	O	—	Human-readable slot label (e.g. `"Pool A"`, `"Round of 32"`)
`active_bout`	integer	O	—	`id` of the currently active bout; matches `match` in `software/match`. Absent if no bout is currently active. **Set only in response to an apparatus-originated `NEXT` or `PREV` control command** — never predicted or advanced by the CMS on its own (e.g. on bout confirmation, or on slot activation). The apparatus is the sole authority for which bout is actually active; the CMS republishing a guess here risks disagreeing with what the apparatus operator actually does next (a `PREV` press, a different bout chosen by dynamic reordering, etc.), leaving display components out of sync with the piste.
`referee`	object	O	—	Primary referee — `{id, name, nation}`, same shape as `common.referee` in `software/fencers`
`referee2`	object	O	—	Second referee — present when the slot has two referees assigned (common in team competitions)
`video_official`	object	O	—	Video review official assigned to this slot
`assessor1`	object	O	—	First assessor assigned to this slot
`assessor2`	object	O	—	Second assessor assigned to this slot
`participants`	array	O	`[]`	Ordered participant list. SHOULD be present for pool rounds to enable matrix rendering. MAY be omitted for DE rounds — fencer identities are available within each bout object.
`participants[].position`	integer	M	—	1-based position in the ordered list; used for matrix row/column ordering
`participants[].id`	string	M	—	Fencer identifier
`participants[].name`	string	M	—	Fencer name
`participants[].nation`	string	O	—	IOC 3-letter nation code
`participants[].club_abbr`	string	O	—	Club abbreviation
`bouts`	array	M	—	Ordered list of bouts in this slot, in official bout order
`bouts[].id`	integer	M	—	Bout identifier — 1-based, local within the slot. Matches `match` in `software/match` for the active bout.

Field	Type	M/O	Default	Description
`bouts[].left`	object	M	—	Left fencer identity — same structure as `left.fencer` in `software/fencers`
`bouts[].right`	object	M	—	Right fencer identity
`bouts[].result`	object	O	`null`	Null or absent until the bout is confirmed (ACK received). Fields: `left_score` (integer), `right_score` (integer), `left_status` (string), `right_status` (string) — same value range as `apparatus/score`.
`bouts[].annotations`	array	O	—	Absent in normal operation. Present only on piste transfer to carry accumulated annotations to the new piste — see below.

Published when

Event	What changes in the payload
Slot assigned to piste (scoresheet `NEXT_SLOT`, see below)	Full payload; all `result` fields null; `active_bout` absent — no bout is active until the apparatus itself requests one
Bout confirmed (ACK)	`bouts[n].result` filled in; `active_bout` reverts to absent — the CMS does not predict what bout comes next; see the `active_bout` field note above
Apparatus `NEXT` / `PREV` (Section 24)	`active_bout` set to the bout the apparatus actually requested
Confirmed fencer swap (Section 15)	`bouts[n].left` / `right` exchanged for the affected bout only — same `slot_id`, same `bouts[n].id`, everything else unchanged. See Section 18 for what the scoresheet does with this.
Piste transfer	Full payload on the new piste's topic — same `slot_id`, same bouts, same results, same `active_bout`; `bouts[n].annotations` populated for completed bouts

Piste transfer

When the CMS reassigns a slot to a different piste (equipment failure, scheduling change), it publishes `software/record` on the **new piste's topic** with the same `slot_id`, the same bouts and results, and any annotations accumulated from `scoresheet/event` messages included in `bouts[n].annotations`. The scoresheet on the new piste bootstraps from this retained message and publishes a matching `scoresheet/record` (Section 18) to re-establish the annotation log on the new piste.

This mirrors the apparatus recovery pattern: the CMS republishes `software/fencers` and `software/match`, and the apparatus loads its current state from those messages. The same principle applies here — the CMS is the authoritative source for the current state of any slot, and display components treat its retained messages as their starting point.

A piste transfer is not assumed to preserve any physical continuity — the new piste may have a different apparatus, a different location, or both. Anything the CMS itself holds (the bout list, results so far, the officiating roster, and — carried over as a known fact, not a new prediction — `active_bout`) is republished as above. Anything only the old apparatus knew (current score, cards, priority, elapsed clock time, UW2F

timer) cannot be recovered from `software/record` alone; the CMS separately seeds the new piste's apparatus with `software/score`, `software/clock`, and `software/uw2f` set to the old apparatus's last-known values, so an in-progress bout resumes rather than restarting from zero. See Sections 11, 13, and 19.

Slot navigation (scoresheet-initiated)

A piste can have more than one slot queued (e.g. several pool rounds scheduled back to back). Activating the next queued slot, or stepping back to the previous one, is initiated by the **scoresheet** — the table official's device, not the scoring apparatus — via `scoresheet/control` (Section 24):

- **`NEXT_SLOT`** requests activation of the next pending slot for this piste. The CMS responds with a `software/record` publish as described in the "Slot assigned to piste" row above (`active_bout` absent). If the currently active slot still has one or more bouts without a confirmed result, the CMS MUST ignore the command and publish nothing — a slot may not be skipped while work on it remains.
- **`PREV_SLOT`** requests reactivation of the previous slot for this piste. The CMS MUST ignore the command and publish nothing unless the currently active slot has zero confirmed bouts (i.e. `bouts[].result` is null for every bout) — stepping back once real results exist would silently undo them, which is a distinct, deliberate operation outside the scope of this command.

Neither command sets `active_bout`, for the same reason ACK does not (see the `active_bout` field note above): which bout is actually loaded is known only once the apparatus itself issues its own `NEXT` / `PREV`.

---

18. Message: scoresheet/record

Topic: `openpiste/{piste_id}/scoresheet/record` **QoS:** 1 **Retained:** Yes

Published by the electronic scoresheet to maintain the authoritative retained log of all annotations made during the current slot. On every new annotation the scoresheet republishes the complete accumulated list — the broker always holds the full history. Any subscriber connecting mid-slot receives the full annotation state immediately.

This message is the complement to `software/record` (Section 17): the CMS owns bout structure and results; the scoresheet owns the annotation log. The `slot_id` field ties the two together.

Relation to `scoresheet/event`: `scoresheet/event` (Section 22) is a per-event fire-and-forget notification for real-time subscribers such as the CMS. `scoresheet/record` is the accumulated retained state. Both are published on every new annotation. Subscribers that only need the current complete state — a late-joining display, or the scoresheet itself on reconnect — subscribe to `scoresheet/record`. Subscribers that must react to each individual event — such as the CMS storing annotations in a database — subscribe to `scoresheet/event`.

Payload

```
{
  "protocol":    "OPP2",
  "version":     "1.0",
  "seq":         58,
  "slot_id":     "f47ac10b-58cc-4372-a567-0e02b2c3d479",
  "annotations": [
    {
      "bout_id": 1,
      "event":   "CARD_REASON",
      "side":    "left",
      "card":    "yellow",
      "reason":  "Corps-à-corps",
      "ts":      1715539200789,
      "official": { "id": "132", "name": "J. Smith", "role": "referee" }
    },
    {
      "bout_id": 1,
      "event":   "CARD_REASON",
      "side":    "right",
      "card":    "red",
      "reason":  "Repeated Group 1: Corps-à-corps",
      "ts":      1715539260123
    }
  ]
}
```

Fields

Field	Type	M/O	Description
`protocol`	string	M	Always `"OPP2"`
`version`	string	M	Protocol version
`seq`	integer	M	Global sequence counter
`slot_id`	string	M	Must match the `slot_id` in the current `software/record`. Used by the scoresheet to detect slot changes.
`annotations`	array	M	Complete ordered list of all annotations for this slot. Empty array `[]` if none recorded yet.
`annotations[].bout_id`	integer	M	Identifies which bout this annotation belongs to. Matches `bouts[].id` in `software/record`.
`annotations[].event`	string	M	Event type — same values as `scoresheet/event`
`annotations[].side`	string	O	`"left"` or `"right"` — for side-specific annotations
`annotations[].card`	string	O	Card type: `"yellow"`, `"red"`, `"black"` — for CARD_REASON events
`annotations[].reason`	string	O	Recorded reason or note
`annotations[].ts`	integer	M	Timestamp of the annotation — see Section 28
`annotations[].official`	object	O	Deciding official — same shape and meaning as `official` in `scoresheet/event` (Section 22)

Scoresheet startup and reconnect sequence

On startup or reconnect, the scoresheet follows this sequence:

1. Subscribe to `software/record` and `scoresheet/record` — the broker delivers both retained messages immediately.
2. Read `software/record` → extract `slot_id`, bout structure, and any initialisation annotations (present only on a piste transfer).
3. Read `scoresheet/record` → compare its `slot_id` with the one from `software/record`.
4. **Match** → restore annotation history from `scoresheet/record`. If `software/record` also carries annotations (piste transfer), merge and deduplicate by `bout_id` + `ts`.
5. **Mismatch or absent** → clear annotation list; publish a fresh `scoresheet/record` with the new `slot_id` and an empty `annotations` array.

Slot change mid-session

When the scoresheet receives a new `software/record` with a different `slot_id`, it clears its annotation list and publishes a fresh `scoresheet/record` with the new `slot_id` and an empty annotations array. The previous slot's annotations remain in the CMS database, where they were stored on receipt of each `scoresheet/event`.

Fencer swap mid-bout

The scoresheet follows the scoring apparatus, not the other way around. When it receives a new `software/record` with the **same** `slot_id` but a `bouts[n].left` / `right` pair that has changed for a `bouts[n].id` it already knows — with everything else in that bout entry unchanged — that is a confirmed fencer swap (Section 15), not a slot change, and the scoresheet MUST NOT clear its annotation list. Software only ever republishes `software/record` this way after validating the swap itself, so the scoresheet does not need to re-validate it.

Instead, the scoresheet exchanges `side` on every existing annotation in its own `scoresheet/record` for that `bout_id` (`"left"` ↔ `"right"`) — so a card recorded against the fencer who was on the left before the swap stays attached to that same fencer, who is now on the right — and republishes its own corrected `scoresheet/record`. This mirrors exactly what software itself does to its own bout/score/card records on the same event (Section 15); the two update independently, driven by the same underlying fact, without either waiting on the other.

---

19. Message: uw2f

Topic: `openpiste/{piste_id}/{publisher}/uw2f` **QoS:** 1 **Retained:** `apparatus/uw2f` — Yes. `software/uw2f` — No, see Section 4.5 for rationale.

Published on any change to the unwillingness-to-fight (passivity) timer or P-card state. The UW2F timer counts upward from zero. `apparatus/uw2f` is QoS 1, not QoS 0 like `apparatus/clock` — it changes only on P-card issuance or an explicit timer transition, not once per second, so there is no next tick to supersede a dropped message and delivery must be guaranteed. Retention addresses a separate need: a newly-connecting or reconnecting subscriber must see the current timer value and P-card counts immediately rather than wait for the next change event — which, especially while the timer sits idle at zero between passivity sequences, may not arrive for a long time. The apparatus publishes `apparatus/uw2f` as the continuously-held, authoritative state for its own display.

Competition management software publishes `software/uw2f` to seed or correct the apparatus's UW2F timer and P-card counts to a specific value — a one-shot instruction, analogous to `software/clock` (Section 11) and `software/score` (Section 13), not a repeating stream. Its case is the same as `software/clock`'s piste-transfer use: when an in-progress bout moves to a different apparatus mid-match, the new apparatus is seeded with the old one's passivity timer and P-cards so they survive the move instead of resetting to zero.

Payload — apparatus/uw2f

```
{
  "protocol": "OPP2",
  "version":  "1.0",
  "seq":      47,
  "time_ms":  60000,
  "time":     "1:00",
  "right": {
    "p_card": 1
  },
  "left": {
    "p_card": 0
  }
}
```

Payload — software/uw2f

```
{
  "protocol": "OPP2",
  "version":  "1.0",
  "seq":      12,
  "time_ms":  60000,
  "time":     "1:00",
  "right": {
    "p_card": 1
  },
  "left": {
    "p_card": 0
  }
}
```

Fields

Field	Type	M/O	Default	Description
`protocol`	string	M	—	Always `"OPP2"`
`version`	string	M	—	Protocol version
`seq`	integer	M	—	Global sequence counter
`time_ms`	integer	M*	`0`	UW2F timer value in milliseconds, counting up from zero
`time`	string	M*	`"0:00"`	UW2F timer formatted as `"M:SS"`
`right.p_card`	integer	M	`0`	Right fencer P-card status — see values below
`left.p_card`	integer	M	`0`	Left fencer P-card status

* At least one of `time_ms` or `time` MUST be present. Implementations MAY include both.

P-card values:

Value	Meaning
`0`	No P-card
`1`	First P-card
`2`	Second P-card
`3`	Third P-card
`4`	Fourth P-card
`5`	Fifth P-card

P-card semantics (which card type corresponds to which ordinal) are defined by the applicable rulebook and may change between rule editions. The protocol records only the ordinal position.

---

20. Message: medical

Topic: `openpiste/{piste_id}/apparatus/medical` **QoS:** 1 **Retained:** Yes

Published when a medical timeout is granted and on every subsequent timer update. The medical timeout is initiated via a `MEDICAL` control command (see Section 24) issued by the apparatus when the referee grants the timeout. The countdown timer runs from the duration specified in the initiating control command.

Payload — timeout active

```
{
  "protocol":     "OPP2",
  "version":      "1.0",
  "seq":          48,
  "active":       true,
  "side":         "left",
  "duration_ms":  300000,
  "remaining_ms": 247000,
  "remaining":    "4:07"
}
```

Payload — timeout ended or cleared

```
{
  "protocol": "OPP2",
  "version":  "1.0",
  "seq":      49,
  "active":   false,
  "side":     "left"
}
```

Fields

Field	Type	M/O	Description
`protocol`	string	M	Always `"OPP2"`
`version`	string	M	Protocol version
`seq`	integer	M	Global sequence counter
`active`	boolean	M	`true` — timeout in progress; `false` — timeout ended or cleared
`side`	string	M	`"left"` or `"right"` — the fencer granted the timeout
`duration_ms`	integer	M when active	Total timeout duration in milliseconds as specified at initiation
`remaining_ms`	integer	M* when active	Remaining time in milliseconds, counting down
`remaining`	string	M* when active	Remaining time formatted as `"M:SS"`

* At least one of `remaining_ms` or `remaining` MUST be present when active. Timer resolution is 1 second.

---

21. Message: video_review

Topic: `openpiste/{piste_id}/{publisher}/video_review` **QoS:** 1 **Retained:** Yes

Published when a video review is requested or resolved. Carries both the current remaining call counts and the full call history for the bout. The apparatus publishes under `apparatus/video_review` when a fencer requests a review; the video referee system publishes under `var/video_review` when resolving a call.

Payload

```
{
  "protocol": "OPP2",
  "version":  "1.0",
  "seq":      50,
  "left": {
    "remaining": 1,
    "calls": [
      {
        "id":      1,
        "round":   1,
        "time_ms": 89250,
        "granted": false,
        "official": { "id": "ref002", "name": "L. Dubois" }
      }
    ]
  },
  "right": {
    "remaining": 2,
    "calls": []
  }
}
```

Fields

Field	Type	M/O	Default	Description
`protocol`	string	M	—	Always `"OPP2"`
`version`	string	M	—	Protocol version
`seq`	integer	M	—	Global sequence counter
`left.remaining`	integer	M	—	Video review calls remaining for left fencer
`left.calls`	array	M	`[]`	History of all video review calls made by left fencer this bout
`right.remaining`	integer	M	—	Video review calls remaining for right fencer
`right.calls`	array	M	`[]`	History of all video review calls made by right fencer this bout

Call history object fields:

Field	Type	Description
`id`	integer	Sequential call identifier, starting at 1
`round`	integer	Round or period in which the call was made
`time_ms`	integer	Stopwatch value in milliseconds at the moment of the call
`granted`	boolean	`true` — granted; `false` — denied. Absent if not yet resolved.
`official`	object	Video official who resolved this call — `{id, name}`. Absent while unresolved.

Initial call counts by phase:

- Pool matches and team matches: 1 call per fencer
- Direct elimination: 2 calls per fencer

These counts reflect current FIE rules and are subject to change. The apparatus or competition management software is responsible for initialising the correct count.

---

22. Message: scoresheet/event

Topic: `openpiste/{piste_id}/scoresheet/event` **QoS:** 1 **Retained:** No

Published by the electronic scoresheet on each individual annotation — a card reason, medical note, reserve fencer introduction, or other table official record. This is a per-event fire-and-forget notification. The broker does not retain it.

Real-time subscribers — primarily the CMS — subscribe to this topic to react to each annotation as it occurs, for example to store it in a database. The CMS SHOULD persist every received annotation so it can include them in `software/record` when transferring a slot to a different piste.

For the full accumulated annotation history — needed by late-joining displays or after a scoresheet reconnect — see `scoresheet/record` (Section 18). Both messages are published on every new annotation: `scoresheet/event` for real-time subscribers, `scoresheet/record` as the retained accumulated state.

Retained rationale: `scoresheet/event` is not retained because it is a point-in-time event notification, not a state description. A retained event would deliver a single annotation to late subscribers with no preceding context — misleading rather than helpful. The full annotation history is always available in `scoresheet/record`.

Payload

```
{
  "protocol":  "OPP2",
  "version":   "1.0",
  "seq":       52,
  "ts":        1715539200789,
  "bout_id":   1,
  "event":     "CARD_REASON",
  "side":      "left",
  "card":      "yellow",
  "reason":    "Corps-à-corps",
  "official": {
    "id":   "132",
    "name": "J. Smith",
    "role": "referee"
  }
}
```

Fields

Field	Type	M/O	Description
`protocol`	string	M	Always `"OPP2"`
`version`	string	M	Protocol version
`seq`	integer	M	Global sequence counter
`ts`	integer	M	Timestamp of the annotation — see Section 28
`bout_id`	integer	O	Identifies which bout in the current slot this annotation belongs to. Matches `bouts[].id` in `software/record`. Absent if the annotation is not bout-specific.
`event`	string	M	Event type — see defined values below
`side`	string	O	`"left"` or `"right"` — for side-specific events
`card`	string	O	Card type: `"yellow"`, `"red"`, `"black"` — for CARD_REASON events
`reason`	string	O	Free text reason or note recorded by the table official
`official.id`	string	O	Identifies which official made this specific decision — distinct from the roster in `software/record`, which only says who is assigned to the bout. Matches the `id` of one of `software/record`'s `referee` / `referee2` / `assessor1` / `assessor2`. Absent if not attributed to a specific official.
`official.name`	string	O	Deciding official's name
`official.role`	string	O	`"referee"`, `"referee2"`, `"assessor1"`, or `"assessor2"` — the deciding official's capacity on this slot

Defined event values

Event	Description
`"CARD_REASON"`	Reason recorded for a card — `bout_id`, `side`, `card`, and `reason` fields apply
`"MEDICAL_NOTE"`	Medical timeout note — `side` and `reason` fields apply
`"RESERVE"`	Reserve fencer introduction recorded — `side` field required

Additional event values may be defined in future revisions without a protocol version change.

---

23. Message: var/connection

Topic: `openpiste/{piste_id}/var/connection` **QoS:** 1 **Retained:** Yes

Indicates whether the video referee system is currently connected and active for this piste. Structured identically to `apparatus/connection` and `software/connection`. The apparatus and scoresheet may watch this topic to know whether a VAR system is present.

Payload — online

```
{
  "protocol":   "OPP2",
  "version":    "1.0",
  "seq":        1,
  "online":     true,
  "software":   "OpenPiste-VAR",
  "sw_version": "1.0.0"
}
```

Payload — offline (LWT)

```
{
  "online": false
}
```

---

24. Message: control

Topic: `openpiste/{piste_id}/{publisher}/control` **QoS:** 1 **Retained:** No

Published when a control event occurs. This topic is bidirectional — it carries commands from apparatus to software, from software to apparatus, from remote controls to apparatus, and from the scoresheet or the video referee system to software. The publisher segment in the topic identifies the source: `apparatus/control`, `software/control`, `remote/control`, `scoresheet/control`, or `var/control`. A receiver that encounters an unknown command value SHOULD ignore it.

Payload

```
{
  "protocol": "OPP2",
  "version":  "1.0",
  "seq":      51,
  "ts":       1715539200456,
  "command":  "MEDICAL",
  "side":     "left",
  "duration": 300
}
```

Fields

Field	Type	M/O	Description
`protocol`	string	M	Always `"OPP2"`
`version`	string	M	Protocol version
`seq`	integer	M	Global sequence counter
`ts`	integer	M	Timestamp when command was issued — see Section 28
`command`	string	M	Command name — see defined values below
`side`	string	O	`"left"` or `"right"` — for side-specific commands
`duration`	integer	O	Duration in seconds — for MEDICAL command only

Defined command values

Command	Publisher	Description
`"NEXT"`	apparatus	Request next match or round
`"PREV"`	apparatus	Request previous match or round
`"END"`	apparatus	Signal end of match or round, awaiting ACK
`"NEXT_SLOT"`	scoresheet	Request activation of the next pending slot for this piste. Ignored if the currently active slot still has unconfirmed bouts — see Section 17
`"PREV_SLOT"`	scoresheet	Request reactivation of the previous slot for this piste. Ignored unless the currently active slot has zero confirmed bouts — see Section 17
`"MEDICAL"`	apparatus	Medical timeout granted; `side` and `duration` required
`"RESERVE"`	apparatus or scoresheet	Reserve fencer introduction; `side` required
`"VIDEO_REVIEW_REQUEST"`	apparatus	Fencer requests video review; `side` required
`"ACK"`	software	Approve end of match or round
`"NAK"`	software	Reject end of match or round
`"VIDEO_REVIEW_GRANTED"`	var	Video review call granted; `side` required
`"VIDEO_REVIEW_DENIED"`	var	Video review call denied; `side` required
`"BEGIN"`	remote	Start the bout
`"HALT"`	remote	Call halt
`"RESET"`	remote	Reset the apparatus
`"VALIDATE"`	remote	Confirm end of match

Additional command values may be defined in future revisions without a protocol version change.

---

25. Apparatus state machine

The state machine described in this section is derived from EFP1.1 (Cyrano protocol, version 1.1, October 2019), Section 4, authored by J-F Nicaud and the Favero Company. It has been adapted to the OPP2 publish/subscribe model: direction is determined by the publisher segment in the topic hierarchy rather than by message type, and "sending" and "receiving" are replaced by "publishing" and "subscribing".

25.1 States

The apparatus operates in one of five states at all times. The current state is published to `apparatus/state` on every transition.

State	Meaning
Waiting (`"W"`)	No active bout. A match may or may not be loaded. The stopwatch may show a scheduled start time. The apparatus is ready to receive match data from software or to begin a bout.
Fencing (`"F"`)	The bout is active and the stopwatch is running. The apparatus publishes clock, lights, score, and blade contact messages as events occur.
Halt (`"H"`)	The bout is paused. The stopwatch is stopped. The apparatus remains in Active state (see Section 25.2).
Pause (`"P"`)	Between periods. The stopwatch is running (counting down the inter-period break).
Ending (`"E"`)	The apparatus has signalled the end of a match or round and is awaiting ACK or NAK from software. The apparatus remains in this state until it receives a response.

25.2 Active and Waiting

The four states Fencing, Halt, Pause, and Ending are collectively referred to as **Active**. The apparatus publishes all changes — score, lights, stopwatch, cards — while in Active state. While in Waiting state, the apparatus publishes only its basic state and any available match information.

25.3 State transition table

Current state	Event	Apparatus behaviour	New state
Active (any)	Any change on the apparatus	Publishes affected topics immediately	Active (unchanged)
Active (any)	Software publishes `software/connection` with `"online": true`	Publishes all current state topics in full	Active (unchanged)
Active (any)	Operator presses END	Publishes `apparatus/control` with `"command": "END"`; publishes `apparatus/state` with `"state": "E"`	Ending
Ending	Software publishes `software/control` with `"command": "ACK"`	Publishes `apparatus/state` with `"state": "W"`	Waiting
Ending	Software publishes `software/control` with `"command": "NAK"`	Returns to previous Active state; MAY display rejection message	Halt
Waiting	Software publishes `software/fencers` and `software/match`	Apparatus loads the match data	Waiting
Waiting	Software publishes `software/connection` with `"online": true`	Publishes current state topics; if no match loaded, publishes `apparatus/control` with `"command": "NEXT"`	Waiting
Waiting	Remote publishes `remote/control` with `"command": "BEGIN"`	Starts the bout; publishes `apparatus/state` with `"state": "F"`	Fencing
Waiting	Operator presses NEXT	Publishes `apparatus/control` with `"command": "NEXT"`	Waiting
Waiting	Operator presses PREV	Publishes `apparatus/control` with `"command": "PREV"`	Waiting
Waiting	Software publishes `software/control` with `"command": "ACK"`	Ignored	Waiting

Notes:

- NEXT and PREV have no effect while the apparatus is in Active state.
- When in Active state, any change (score, lights, cards, clock) triggers immediate publication of the affected topic.
- On receipt of a `software/fencers` or `software/match` message, the apparatus updates its display but does not change state or reset scores. This mirrors EFP1.1 DISP behaviour.

25.4 Correct and incorrect end of match

The apparatus evaluates whether the end of a match is formally correct before publishing the END command. This evaluation applies to individual competitions; team round endings are always considered correct unless it is the final round.

Correct ending — one of the following must be true:

- Both fencer statuses are normal (`"U"` or `"V"` / `"D"`) AND the scores are different
- Both fencer statuses are normal AND the scores are equal AND a priority is assigned (`"R"` or `"L"`)
- At least one fencer has status `"A"` (abandonment) or `"E"` (exclusion)

Incorrect ending — both fencer statuses are normal, scores are equal, and priority is `"N"`. The apparatus SHOULD NOT publish the END command in this situation, or MAY publish it and expect a NAK response from software.

When software responds with NAK, the apparatus returns to Halt state and SHOULD display an appropriate message to the operator (e.g. "END not accepted").

Software has one additional, independent reason to NAK, beyond the score/status/priority checks above, which the apparatus itself has no way to evaluate: an unresolved `apparatus/fencers` mismatch (Section 15) — a fencer-identity update that wasn't a clean left/right exchange of the currently assigned pair. Software MUST NAK an END in this state regardless of how correct the score/priority otherwise looks, since it cannot yet be sure which fencer the recorded scores belong to.

25.5 Score publishing behaviour

The apparatus publishes `apparatus/score` on every score change and also when it first loads a match. When software sends a `software/match` message containing pre-existing scores (e.g. for team rounds after the first, or when restoring a previous match), the apparatus displays those scores immediately. The BEGIN command does not reset scores that were supplied via `software/match`.

25.6 Clock publishing behaviour

The apparatus publishes `apparatus/clock` once per second while the stopwatch is running, and immediately on any clock state change (start, stop, reset). More frequent publication is unnecessary and SHOULD be avoided — at peak competition load a broker may be serving many pistes simultaneously.

25.7 Reserve fencer

When the referee introduces a reserve fencer, the apparatus publishes `apparatus/control` with `"command": "RESERVE"` and `"side": "left"` or `"right"`. This signal is valid only in team competitions when a reserve fencer has been declared and has not yet been used. Software responds by updating the fencer assignment for subsequent rounds via `software/fencers`.

---

26. Field types and conventions

26.1 JSON types

Type	JSON representation	Notes
Boolean	`true` / `false`	Never `"0"` / `"1"` or string-encoded
Integer	JSON number, no quotes	Scores, card counts, millisecond times, sequence counter
String	JSON string	Identifiers, names, nation codes, formatted times
Timestamp	JSON integer (64-bit)	See Section 28 for encoding convention

Formatted time strings use `"M:SS"` or `"M:SS.cc"` format. Hundredths are mandatory when time is below 10 seconds, consistent with EFP1.1 convention.

Nation codes use IOC 3-letter codes (e.g. `"FRA"`, `"GBR"`, `"ITA"`).

String encoding. All string fields are UTF-8 encoded JSON strings, per RFC 8259. OPP2 natively supports non-ASCII characters in `name` fields — diacritics (e.g. "François", "Łukasz") and non-Latin scripts (Cyrillic, Chinese, etc.) are valid and have been confirmed working in practice with the reference ArduinoJson implementation.

The `nation` field is the one exception: it MUST remain restricted to standard IOC 3-letter ASCII codes (uppercase A–Z only). Downstream systems — flag icon lookups, results databases, broadcast graphics — key off the exact ASCII code, and a non-standard value will silently break those integrations even though the JSON itself would parse correctly.

26.2 Mandatory and optional fields

Each field in the per-message tables is marked **M** (mandatory) or **O** (optional).

Mandatory fields MUST be present in every message published by a sender claiming compliance with the declared `version`. A receiver that receives a message with a missing mandatory field SHOULD treat it as a protocol error and MAY discard the message. Mandatory fields that carry a default value in the table have that default defined for receiver use only — a compliant sender must still include the field.

Optional fields MAY be absent. When absent, the receiver MUST apply the default value shown in the table. Optional fields with no default (shown as —) have no meaningful default and their absence simply means the information is unavailable; receivers MUST handle this gracefully.

26.3 Versioning and field obligations

Which fields are mandatory depends on the protocol version the sender declares in the `version` field. A receiver encountering a sender running an older version MUST apply defaults for any mandatory fields that are absent.

- **Receiver knows v1.0, sees v1.0** — enforce mandatory fields strictly.
- **Receiver knows v1.1, sees v1.0** — apply defaults for fields added in v1.1.
- **Receiver knows v1.0, sees v1.1** — accept the message; ignore unknown fields.

- **Receiver encounters an unknown protocol version** — accept permissively; apply defaults for absent fields.

---

27. Sequence counter and idempotency

27.1 Purpose

MQTT QoS 1 guarantees at-least-once delivery, which means a message may be delivered more than once under certain network conditions. Consumers that perform irreversible actions on receipt — updating a score, issuing a command, recording a video review — must be able to detect and discard duplicate deliveries without processing them twice.

27.2 The seq field

Every QoS 1 message carries a mandatory `seq` field: an unsigned 32-bit integer that is incremented by the producer before every publish, regardless of topic. The counter is global — shared across all topics published by one device. It is not reset between topics, only on device reboot.

Using a single global counter means that no two messages from the same device will share the same `seq` value within a session, satisfying per-topic uniqueness as a stronger property. It also allows consumers to reconstruct the cross-topic publish order if needed.

27.3 Detecting duplicates

A consumer tracks the last seen `seq` value per producer (identified by piste ID and publisher segment). If a received message carries a `seq` value already seen from that producer, the message is a duplicate and SHOULD be discarded.

27.4 Detecting a new session after reboot

On device reboot the counter resets to a low value (typically 1). A consumer distinguishes a reboot from a wraparound by checking the timestamp:

- If `seq` resets to a low value AND `ts` has advanced significantly → new session; reset the tracked counter
- If `seq` wraps from near `0xFFFFFFFF` to near `0` AND `ts` is continuous → wraparound, not a reboot

27.5 Counter wraparound

The 32-bit unsigned counter wraps around after approximately 4.3 billion publishes. At one publish per second this takes over 136 years. Wraparound is not a practical concern but consumers SHOULD handle it gracefully as described above.

27.6 QoS 0 messages

`seq` is absent on QoS 0 messages (clock, blade_contact). These messages are inherently lossy by design — the timestamp serves as the primary identity reference for the rare cases where ordering or deduplication matters.

---

28. Timestamp conventions

28.1 UTC only

All timestamps in Level 2 are UTC. No local time, no timezone offsets, no daylight saving adjustments. Unix epoch milliseconds are by definition UTC — this is not a configuration choice, it is inherent to the format. Implementations MUST use UTC time sources and MUST NOT apply local timezone conversions.

28.2 Format

All timestamps are 64-bit unsigned integers. The upper byte (bits 63–56) carries a clock source flag. The lower 56 bits carry the time value in milliseconds.

Bits	Field
63–56	Clock source flag (upper byte)
55–0	Time value in milliseconds (lower 56 bits)

28.3 Flag values

Upper byte	Meaning	Lower 56 bits
`0x00`	NTP — UTC wall clock	Unix epoch milliseconds, NTP synchronised
`0x01`	Session — boot relative	Milliseconds since device boot (`millis()`)
`0x02` – `0xFF`	Reserved	—

28.4 NTP timestamps

Current Unix epoch milliseconds are approximately 1.7×10^{12} (`0x0000018E...` in hex). The upper byte is naturally `0x00` for the foreseeable future. NTP timestamps therefore require no manipulation at the apparatus — the raw epoch millisecond value is correct.

28.5 Session timestamps

When NTP is unavailable, the apparatus SHOULD use milliseconds since device boot with the upper byte set to `0x01`:

```
// NTP available — upper byte is naturally 0x00
uint64_t ts = (uint64_t)epochMillis;

// NTP not available
uint64_t ts = ((uint64_t)0x01 << 56) | (uint64_t)millis();
```

Session timestamps are useful for relative timing within a session but cannot be compared across devices or to wall-clock time.

28.6 Reading timestamps

```
uint8_t  flag = (ts >> 56) & 0xFF;
uint64_t time = ts & 0x00FFFFFFFFFFFFFF;
// flag == 0x00: time is UTC Unix epoch milliseconds
// flag == 0x01: time is milliseconds since device boot
```

28.7 Video synchronisation

When using blade contact or lights timestamps to synchronise video overlays, both the apparatus and the video system SHOULD be synchronised to the same NTP server. Residual clock drift between devices is typically under 10ms on a well-managed local network.

---

29. Versioning and compatibility

29.1 Protocol identifier and version

Every message carries two mandatory fields:

- `"protocol": "OPP2"` — the protocol family identifier. Fixed for all Level 2 messages.
- `"version": "1.0"` — the protocol version as a `"major.minor"` string.

A receiver SHOULD check the `protocol` field and MAY ignore messages with an unrecognised identifier. The `version` field governs which fields are mandatory — see Section 26.2 and 26.3.

29.2 Minor revisions — adding fields

New fields may be added to any message in a minor revision (e.g. `"1.0"` → `"1.1"`). Receivers that know only the older version will encounter unknown fields, which JSON parsers silently ignore — existing receivers continue to operate correctly.

29.3 Breaking changes

Removing or renaming existing mandatory fields, or changing field types, constitutes a breaking change and requires a new protocol identifier (e.g. `"OPP3"`). The `version` field resets to `"1.0"` with each new protocol identifier.

29.4 Adding enumerated values

New values for `command`, `phase_type`, and the `{publisher}` topic segment are not breaking changes and do not require a version increment. Receivers that encounter unknown values SHOULD ignore them.

---

30. Security and provisioning

> **Draft.** This section replaces the previous placeholder with a fully worked design. See the accompanying pull request description for the reasoning trail behind each decision below.

30.1 Trust model

Level 2 assumes a physically-secured local competition network as the outer perimeter — this section does not attempt to defend against a hostile public network, and does not require cryptographic strength beyond what a human-supervised bootstrap ritual plus TLS already provides. Every provisioning path in this section traces back to an operator taking an action that vouches for a new component, the same way physically deploying an apparatus at a venue already does. This is a deliberate, stated bar, not an oversight: implementers SHOULD NOT build additional cryptographic machinery beyond what this section specifies, and MUST NOT assume physical/network access alone is sufficient without it.

30.2 Asymmetric access model

Subscribing is unauthenticated. Any client MAY connect and subscribe to `openpiste/#` without credentials. Publishing MUST be authenticated, gated per the publisher-scoped model below.

30.3 Publisher-scoped access control

Each publisher role MUST be restricted, at the broker, to publishing only within its own topic namespace:

Publisher	Permitted publish namespace
`apparatus`	`openpiste/+/apparatus/#`
`software`	`openpiste/+/software/#`
`remote`	`openpiste/+/remote/#`
`var`	`openpiste/+/var/#`
`scoresheet`	`openpiste/+/scoresheet/#`

This prevents a misconfigured or compromised remote control from publishing score corrections, prevents software from spoofing apparatus connection state, and prevents a scoresheet from publishing video review decisions. How a broker implements this restriction internally (a static ACL file, a plugin, a database-backed check) is not specified — only the restriction itself is normative.

30.4 Provisioning: two tiers by device capability

A component earns the credential it authenticates with via a provisioning exchange, not by manual broker configuration alone. Provisioning always requires an operator action (Section 30.1) — there is no zero-human-involvement path, by design.

Two tiers exist because device capability genuinely differs, not because of a compliance shortcut. Both are legitimate; neither is a fallback for the other:

- **Tier A** — components able to present a TLS client certificate at connection time (embedded firmware, native applications). Provisioned via a fully scripted MQTT exchange (30.5), no manual OS-level step beyond initial physical setup.
- **Tier B** — components unable to present a TLS client certificate (browser-based components — no browser exposes an API for a web page to select or supply a client certificate at connection time). Provisioned via an out-of-band credential exchange (30.6) that necessarily includes at least one manual step.

Tier is a property of what a given implementation can do, independent of which publisher role it fills — a native scoresheet application is Tier A; a browser-based apparatus display, were one ever built, would be Tier B. A component capable of Tier A MAY always use it regardless of role.

30.5 Tier A: certificate-based provisioning

Credential. A TLS client certificate, signed by the deployment's own CA. The requesting component MUST generate its own key pair locally and submit only a certificate signing request (CSR) — the private key MUST NOT be transmitted.

Topics:

```
openpiste/_provision/request
openpiste/_provision/response/{device_id}
```

`_provision` is a reserved pseudo-`piste_id` and MUST NOT be used as a real one. `{device_id}` is a client-generated opaque correlation id. Both topics: QoS 1, not retained (Section 4.5's rationale for `control` applies identically here — a one-shot exchange, not state).

Request (published by the new component):

```
{
  "protocol": "OPP2", "version": "1.0", "seq": 1, "ts": 1715539200000,
  "code": "482913",
  "role": "apparatus",
  "device_id": "b6a1c2d3-...",
  "device_label": "OpenPiste-ESP32",
  "csr": "-----BEGIN CERTIFICATE REQUEST-------..."
}
```

Field	M/O	Description
`code`	M	Operator-issued ticket code (out of protocol scope how it was generated or relayed — an operator-authenticated CMS action)
`role`	M	Publisher role being requested — `apparatus` | `scoresheet` | `remote` | `var`. Never `software` — the CMS is the provisioning authority, not something provisioned.
`device_id`	M	Client-generated opaque id; also the response topic's correlation segment
`device_label`	O	Human-readable description
`csr`	M	PEM-encoded certificate signing request

Response (published by the CMS):

```
{ "protocol": "OPP2", "version": "1.0", "seq": 2, "ts": 1715539201000,
  "status": "granted", "role": "apparatus",
  "cert": "-----BEGIN CERTIFICATE-------...", "ca_cert": "-----BEGIN CERTIFICATE-------..." }
```

Failure: `{"status": "denied", "reason": "invalid_or_expired_code"}`. A signed certificate is not confidential — broadcasting it to any subscriber discloses nothing usable without the private key, which never left the device — so no additional access restriction on the response topic is required.

Revocation. A compliant deployment MUST support revoking a Tier A component's access. Revocation SHOULD be implemented via a certificate revocation list (CRL) checked at the TLS handshake, not via a live responder (OCSP) — a static, occasionally-reloaded file is sufficient, matching the non-urgent nature of revocation generally (Section 30.6). Certificates SHOULD carry a bounded validity period (deployment-scale, not decades) as a complementary, non-exclusive safeguard — so an unrevoked but lost or decommissioned device still stops working within a bounded window.

30.6 Tier B: credential-based provisioning

Credential. An MQTT username/password pair, unique per device — never a shared credential, and never a certificate (30.4). A device's compromise is confined to that device alone, individually revocable, and directly attributable.

Provisioning does not require a broker capable of live credential creation. An operator knows, before a competition, approximately how many Tier B devices will need pairing. A CMS MAY pre-generate a batch of N credentials ahead of time via a static mechanism (e.g. a password file), tracking assignment in its own records. What must be immediate is *assignment* of an already-existing credential to a device, not

its *creation* — this is a CMS-local operation with no broker interaction at pairing time. Revocation (disabling one entry) MAY be a static, occasionally-reloaded operation, consistent with 30.5's Tier A revocation model — revocation is rare and exceptional, not latency-sensitive the way pairing is.

Delivery is out-of-band — not a network exchange. The assigned credential MUST be conveyed to the device via a channel that does not traverse the broker or a cross-network API — e.g. a QR code scanned by the device's camera, or manual entry. This is a direct consequence of the credential being a secret with no equivalent to Tier A's "a certificate is safe to broadcast" property: MQTT pub/sub would expose it to any subscriber, and even a point-to-point network channel (HTTP) would still require the requesting component and the issuing CMS to negotiate a shared origin or CORS policy, which is not guaranteed across vendors. An out-of-band, human-supervised channel avoids both problems at once and requires no new infrastructure beyond what Section 30.4's operator-vouches-for-the-device model already assumes. The exact payload format (e.g. a URI-style QR payload) is not yet finalised.

Accepted, not solved:

- The delivered credential is long-lived, not a short-lived ticket — a photograph of the QR captures a working credential for as long as it remains valid. This is accepted given the operator's control over when and how long it is displayed, not because the exposure is zero.
- At-rest storage on the device (e.g. browser `localStorage`) is plaintext, readable by anything with access to that specific device. No mechanism in this section addresses this — it is a platform property of Tier B devices, not a protocol gap. Per-device credentials (above) bound the *consequence* of this floor; they do not remove it.

Operational recommendations (not normative requirements): operators SHOULD dismiss a credential-bearing display promptly once shown; devices holding a Tier B credential SHOULD use their platform's own lock screen when not in active use. Both raise the bar against casual physical access, the realistic threat model here.

30.7 Capability signaling

A component or broker MAY advertise support for this section via an additional field on its existing `connection` message (Sections 8, 9) — e.g. `"provisioning_tiers": ["A", "B"]`. Absent, this means no support for this section; implementations that predate it remain fully compliant at the access level Sections 30.1–30.3 describe as a baseline, without provisioning support. This section MUST NOT be treated as a breaking requirement for existing deployments.

30.8 What this section does not specify

- How an operator authenticates to a CMS to generate a provisioning ticket, or the ticket's own transport to the operator — entirely a CMS's own concern, no more standardized than how a CMS's admin panel works today.
- Per-piste or per-instance authorization scoping beyond the per-role model in 30.3 — implementations MAY add finer scoping; this section does not require it.
- How a broker internally stores or manages credentials (a plugin, a config file, a database) — only the wire-level credential shape (client certificate, or MQTT username/password) and the Tier A exchange in 30.5 are normative.

---

31. Cloud bridging and competition identity

31.0 Introduction

A local MQTT broker is designed for one venue. It serves the apparatus, the displays, the remote controls, and any other devices physically present at the competition. It is fast, self-contained, and requires no internet connection. This is the right architecture for real-time scoring on the piste.

But there are compelling reasons to make that data available beyond the venue:

Live results from anywhere. Coaches in the warm-up area, team officials in the stands, remote spectators following online — all could benefit from live scoring data if it were accessible outside the venue network.

Federation ranking and results reporting. National and international federations require competition results for ranking calculations, athlete licensing, and official records. Today this typically involves manual export from the CMS and upload to a federation portal after the competition. A cloud broker receiving live data from the venue could feed federation systems directly — results available the moment a match ends, without manual intervention. This applies at every level: club results to national federations, national competition results to continental confederations (EFC, Pan-American Confederation, etc.), and major event results to the FIE.

Video referee synchronisation. A cloud-accessible timestamp stream allows video referee tools running on external infrastructure to synchronise overlays with the live feed without being on the local network.

Archiving and analytics. Every bout, every touch, every card, every clock tick — published with millisecond timestamps — is a rich dataset for performance analysis, referee training, and rule development. Archiving this data to cloud storage requires relaying it from the local broker.

Multi-venue aggregation. A national federation running events simultaneously at multiple venues could aggregate live results from all of them into a single dashboard, without requiring any coordination between venues.

Broadcasting and media. A live results feed accessible via a standard MQTT subscription — or via a cloud-hosted web interface consuming it — gives broadcasters and media organisations a reliable data source without requiring venue access.

Bridging is the mechanism that makes all of this possible without changing anything at the local level. A bridge is a piece of software that subscribes to the local broker and republishes the messages to a cloud broker, enriching the topic with enough context to make the data meaningful and discoverable outside the venue. The local apparatus, the CMS, the displays — none of them need to know the bridge exists.

31.1 Local simplicity by design

A local broker serves one venue. The apparatus knows only that it is on piste 17. It publishes to `openpiste/17/apparatus/lights`. It does not know or care whether that data is consumed locally, relayed to a cloud broker, or archived. This simplicity is intentional and preserved by design.

Cloud connectivity is handled entirely by a **bridge** — a component that subscribes to the local broker and republishes to a cloud broker with an enriched topic prefix. No changes are required to the apparatus, the CMS, or any local subscriber.

Importantly, this bridging capability is built directly into Mosquitto and most other standards-compliant MQTT brokers. No additional middleware or custom software is required to relay messages from a local broker to a cloud broker — it is a native feature, configurable in a few lines. This was an explicit reason for choosing MQTT as the transport for OPP2: the cloud relay capability comes for free with the technology, rather than requiring a bespoke integration layer.

31.2 Cloud topic structure

When relaying from a local broker to a cloud broker, the bridge prepends a structured prefix to every topic:

```
openpiste/{country}/{year}/{month}/{day}/{tournament_id}/{competition_id}/{piste_id}/{publisher}/
```

Segment	Description	Example
`openpiste`	Fixed platform prefix	—
`{country}`	Host country — IOC 3-letter code	`BEL`
`{year}`	Tournament start year — four digits	`2026`
`{month}`	Tournament start month — two digits, zero-padded	`06`
`{day}`	Tournament start day — two digits, zero-padded	`15`
`{tournament_id}`	Machine-readable tournament identifier	`bel-nat-champ-2026`
`{competition_id}`	Machine-readable competition identifier	`efj-eq`
`{piste_id}`	Piste identifier, as used locally	`17`
`{publisher}`	Publisher role, as used locally	`apparatus`
`{message_type}`	Message type, as used locally	`lights`

So a local topic `openpiste/17/apparatus/lights` becomes:

```
openpiste/BEL/2026/06/15/bel-nat-champ-2026/efj-eq/17/apparatus/lights
```

on the cloud broker.

31.3 Rationale for the topic structure

Global uniqueness without a global registry. No single segment needs to be globally unique on its own. The combination of country, date, tournament identifier, and competition identifier creates global uniqueness through hierarchy — the same way a postal address works. `bel-nat-champ-2026/efj-eq` only needs to be unique within `BEL/2026/06/15/`, which is a very small namespace. A national federation can maintain its own list of competition identifiers without coordinating with any global authority.

The date is the tournament start date. For multi-day events, all data lives under the start date regardless of which day a specific bout takes place. This keeps the topic path stable and predictable for the lifetime of the tournament.

Every segment is an independent filter dimension. MQTT wildcard subscriptions match whole segments. By giving country, year, month, and day each their own segment, any combination can be subscribed to independently:

```
openpiste/#                                 # everything, everywhere
openpiste/BEL/#                             # everything in Belgium
openpiste/+/2026/#                          # everything in 2026
openpiste/+/2026/06/#                       # everything in June 2026
openpiste/+/2026/06/15/#                    # everything on June 15th 2026
openpiste/BEL/2026/06/15/#                 # Belgium, June 15th 2026
openpiste/+/+/+/+/bel-nat-champ-2026/#    # one tournament regardless of date
openpiste/+/+/+/+/+/efj-eq/#              # all junior men's épée worldwide
openpiste/+/+/+/+/+/+/17/#               # piste 17 at every venue in the world
```

Note: MQTT wildcards use `+` (single segment) and `#` (all remaining segments). The `*` character is not a valid MQTT wildcard.

Local topics are unchanged. The bridge adds the prefix on relay. The apparatus, CMS, and local subscribers are entirely unaware of the cloud topic structure.

The `ext_id` field complements, not replaces, the topic hierarchy. The topic hierarchy handles routing and discovery on the broker. The `ext_id` field in the identity message (Section 31.5) handles authoritative cross-system identity — linking a competition to the FIE database, a national federation's results system, or any other external registry. These are separate concerns handled by separate mechanisms.

Identifiers SHOULD be lowercase, URL-safe, and use hyphens as word separators. They MUST NOT contain spaces, slashes, or wildcard characters (`#`, `+`).

31.4 Multiple competitions at one venue

A large championship runs multiple weapon and category events simultaneously on different piste groups. Each event is a separate `competition_id` under the same `tournament_id`. Piste numbers are assigned locally and may overlap between competitions — this is not a problem because the full topic path is unambiguous.

```
openpiste/ITA/2026/06/15/eur-champ-2026/efj-eq/17/apparatus/score   # junior men's épée, piste 17
openpiste/ITA/2026/06/15/eur-champ-2026/esf-foil/17/apparatus/score # senior women's foil, piste
```

31.5 Competition identity message

The bridge publishes a retained identity message to the cloud broker at the start of each competition. This serves as the discovery layer — a subscriber can watch `openpiste/+/+/+/+/+/+/identity` to find all currently live competitions on the cloud broker, or narrow the subscription to a specific country or date

range.

Topic: `openpiste/{country}/{year}/{month}/{day}/{tournament_id}/{competition_id}/identity`
QoS: 1 **Retained:** Yes

```
{
  "protocol": "OPP2",
  "version":  "1.0",
  "tournament": {
    "id":         "eur-champ-2026",
    "name":       "European Fencing Championships 2026",
    "city":       "Genova",
    "country":    "ITA",
    "start_date": "2026-06-15",
    "end_date":   "2026-06-22",
    "organiser":  "EFC",
    "ext_id":     "EFC-2026-007"
  },
  "competition": {
    "id":       "efj-eq",
    "name":     "Junior Men's Épée Individual",
    "weapon":   "E",
    "type":     "I",
    "category": "Junior",
    "gender":   "M"
  }
}
```

31.5 Identity message fields

Tournament fields:

Field	Type	M/O	Description
`id`	string	M	Machine-readable tournament identifier, matching the topic segment
`name`	string	M	Full human-readable tournament name
`city`	string	M	Host city
`country`	string	M	Host country — IOC 3-letter code
`start_date`	string	M	Tournament start date as `"YYYY-MM-DD"`
`end_date`	string	M	Tournament end date as `"YYYY-MM-DD"`
`organiser`	string	O	Organising body (e.g. `"FIE"`, `"EFC"`, club name)
`ext_id`	string	O	External identifier for linking to federation databases

Competition fields:

Field	Type	M/O	Description
`id`	string	M	Machine-readable competition identifier, matching the topic segment
`name`	string	M	Full human-readable competition name
`weapon`	string	M	`"F"` foil, `"E"` épée, `"S"` sabre
`type`	string	M	`"I"` individual, `"T"` team
`category`	string	O	Age category (e.g. `"Senior"`, `"Junior"`, `"U17"`)
`gender`	string	O	`"M"` men, `"F"` women, `"X"` mixed

31.6 Bridge configuration and CMS integration

> **Open item — input from CMS developers and competition organisers is actively sought.**

The bridge requires two pieces of information to operate: the `tournament_id` and `competition_id`. How this information reaches the bridge is an open architectural question.

Option A — Manual bridge configuration. The bridge operator manually enters the tournament and competition identifiers at setup. Simple and reliable, but requires human action and introduces a configuration step separate from the CMS.

Option B — CMS-driven configuration. The CMS publishes competition metadata to a well-known local topic when a competition is loaded. A bridge-side component subscribes to this topic and automatically configures the cloud routing. This is the cleanest model — the CMS continues to do its job (managing competitions) without needing to know about MQTT or cloud infrastructure. However it requires the CMS to publish this metadata, which currently no CMS does.

Option C — Bridge monitors the local identity topic. A simpler variant of Option B: the CMS or a middleware component publishes to `openpiste/identity` on the local broker. The bridge watches this topic and updates its prefix configuration automatically.

The preferred long-term solution is Option B or C — keeping the CMS unaware of cloud infrastructure while enabling automatic configuration. The right answer depends on how CMS software is structured in practice. Feedback from CMS developers (EnGarde, Engarde, Fencing Time, national federation systems) and competition organisers who have deployed MQTT infrastructure is welcome at https://github.com/OpenPiste/protocol/issues.

---

32. Open items

ACK/NAK state machine. The full state machine around the Ending state — particularly the exact behaviour when NAK is received mid-bout versus at the end of a team round — is not yet formally specified beyond what is covered in Section 25.

JSON Schema. A machine-readable JSON Schema for all message types is planned as a separate document at `schemas/opp2/` in the OpenPiste repository. Not yet published.

Cloud bridge CMS integration. How the bridge obtains tournament and competition identifiers from the CMS without requiring CMS-side MQTT awareness is not yet resolved. See Section 31.6.
